Project - Testing

Presented by:

Daniel Libardo Diaz Gonzalez - *dandiazgo@unal.edu.co*
Andrés Felipe León Sánchez - *anleonsa@unal.edu.co*
Alejandro Medina Rojas - *alemedinaro@unal.edu.co*
Angel David Ruiz Barbosa - *aruizba@unal.edu.co*

Professor:

Oscar Eduaro Alvarez Rodriguez
oalvarezr@unal.edu.co

Saturday 19th of July



Universidad Nacional de Colombia
Facultad de Ingeniería
Ingeniería de Sistemas y Computación
2025



CONTENT

1. Unit Testing:..... 3
2. Linter:.....4
3. Linter screenshoots:..... 8

1. Unit Testing:

For testing our project, we used the Google Test tool. Google Test is a framework focused on the development of unit tests in C++. This tool can be easily integrated with CMake and also offers high portability across different operating systems.

This framework is particularly effective due to its ease of use and the fact that it is an open-source project with a large and active community. It is worth noting that Google Test specializes in performance testing, as is often the case with tools applied to C++.

However, among its main disadvantages is the fact that it significantly increases compilation time when building the project. Additionally, Google Test is not very useful when performing tests involving the graphical user interface (GUI).

Pruebas: A total of 47 different tests were carried out, focused on the various actions of the "File" menu, as this was one of the features that presented the most issues.

Below are the 12 most significant tests.

1) NewActionClearsAllEditorsAndResetsTitle:
   The previous test verifies that the "New file" action correctly resets the working environment. It does so by checking that, after initializing a new document, the application's title is reset and the editors are cleared.
2) SaveSpecialCharacters:
   This test checks that special characters are preserved after saving the document.
3) SaveAsActionUpdatesWindowTitle:
   The purpose of this test is to ensure that the window title reflects the name of the saved file.
4) SpecialCharactersHandling:
   The test ensures that files containing complex Unicode characters are read correctly.
5) ExitActionConnectedToQuit:
   As the name suggests, this test verifies that the application closes after triggering the Exit action.
6) NewActionWorksWhenEditorsAlreadyEmpty:
   This test validates the stability of executing "New" when the editors are already empty.
7) SaveToFileWithSpacesInName:
   SaveToFileWithSpacesInName verifies that files with special characters in their names are saved correctly.
8) SaveAsActionPreservesEditorContent:
   This test confirms that the "Save as" action does not alter the editor's content.

9) FilePathWithSpaces:

This test validates that files located in paths containing spaces can be opened correctly.

10) ExitActionStateUnaffectedByEditorContent:

It verifies that the "Exit" action is not affected by the content of the editor.

11) OverwriteExistingFile:

This test validates the overwriting of existing files.

12) LargeFileHandling:

Finally, this test validates that the editor functions correctly even when handling files that occupy a large amount of space.

Imágen de la raíz del proyecto con la carpeta de los tests abiertos



2. Linter:

The linter tool is divided into two sub-tools that assist with static code analysis during application development: the formatter and the static code analyzer.

Role of the formatter: Defines a set of formatting rules that the code must follow.
Role of the analyzer: Defines a set of coding practice rules that the code must follow.

Formatter used: clang-format
Analyzer used: clang-tidy

Rules defined for the formatter:

- Based on the LLVM style as the starting point.
- Indentation with 4 spaces (tabs are not used).
- Braces are placed on the same line as the control statement (if, for, etc.).
- Short functions may be placed on a single line if they fit.
    - and & bind to the variable name, not the type.
- A space is added before the opening parenthesis in control statements (if, for, etc.).
- No spaces inside <...> template argument lists.
- No spaces inside initializer lists like {1, 2} or [1, 2].
- A space is added around assignment operators (a = b).
- Standard spacing around type qualifiers like const or volatile.
- There is a space after a C-style cast: (int) x → (int) x, not (int)x.
- No spaces are allowed immediately before end-of-line comments.
- No space is added before C++11 brace initialization lists (MyType x{1, 2} → no space before {).
- Use modern brace-list ({}) initialization style for C++11.
- Trailing whitespace at the end of a line is not allowed.
- Template declarations are not split across multiple lines.
- Always add a single blank line at the end of each file.
- Very wide column limit: 1000 characters.



Code of the formatter rules implemented in the .clang-format file at the root of the project:

```
BasedOnStyle: LLVM
IndentWidth: 4
UseTab: Never
BreakBeforeBraces: Attach
AllowShortFunctionsOnASingleLine: Inline
PointerAlignment: Right
SpaceBeforeParens: ControlStatements
SpacesInAngles:  false
SpacesInContainerLiterals: false
SpaceBeforeAssignmentOperators: true
SpaceAroundPointerQualifiers: Default
SpaceAfterCStyleCast: true
SpacesInSquareBrackets: false
SpaceBeforeCpp11BracedList: false
SpacesBeforeTrailingComments: 1
Cpp11BracedListStyle: true
AllowTrailingWhitespace: false
AlwaysBreakTemplateDeclarations: No
InsertTrailingNewline: true
ColumnLimit: 1000
```

Rules defined for the checker.

Checks:

1. All default checks are disabled (-*).
2. All checks that detect common or dangerous bugs are enabled (bugprone-*).
3. Checks that detect performance issues are enabled (performance-*).
4. Checks that improve code readability are enabled (readability-*).
5. Checks that suggest using modern C++ features are enabled (modernize-*).

6. The modernize-use-trailing-return-type check is excluded (to avoid suggestions like auto func() -> int).

7. The C++ Core Guidelines checks are enabled (cppcoreguidelines-*).

8. The check that discourages the use of non-const global variables is excluded (cppcoreguidelines-avoid-non-const-global-variables).

9. The google-readability-function-size check is enabled to warn about overly long functions.

10. The readability-identifier-naming check is enabled to enforce strict rules on identifier names.

Name casing:

1. Functions must use camelCase (e.g., MyFunction).

2. Methods (functions inside classes) must also use camelCase.

3. Classes must use PascalCase (e.g., MyClass).

4. Structs must also use PascalCase.

5. Namespaces must use lower_case (e.g., utilities, red).

6. Regular variables must use snake_case (e.g., max_quantity, element_size).

7. Global constants must use UPPER_CASE (e.g., MAX_BUFFER_SIZE).

Código de las reglas del formateador implementadas en el archivo .clang-format en la raíz del proyecto:

```
Checks: >
  -*,
  bugprone-*,
  performance-*,
  readability-*,
  modernize-*,
  -modernize-use-trailing-return-type,
  cppcoreguidelines-*,
  -cppcoreguidelines-avoid-non-const-global-variables,
  google-readability-function-size,
  readability-identifier-naming,
```

```
CheckOptions:
  - key:
readability-identifier-naming.FunctionCase
    value:           camelBack
  - key:             readability-identifier-naming.MethodCase
    value:           camelBack
  - key:             readability-identifier-naming.ClassCase
    value:           PascalCase
  - key:             readability-identifier-naming.StructCase
    value:           PascalCase
  - key:
readability-identifier-naming.NamespaceCase
    value:           lower_case
  - key:
readability-identifier-naming.VariableCase
    value:           snake_case
  - key:
readability-identifier-naming.GlobalConstantCase
    value:           UPPER_CASE
```

3. **Linter screenshoots:**

clang-format:

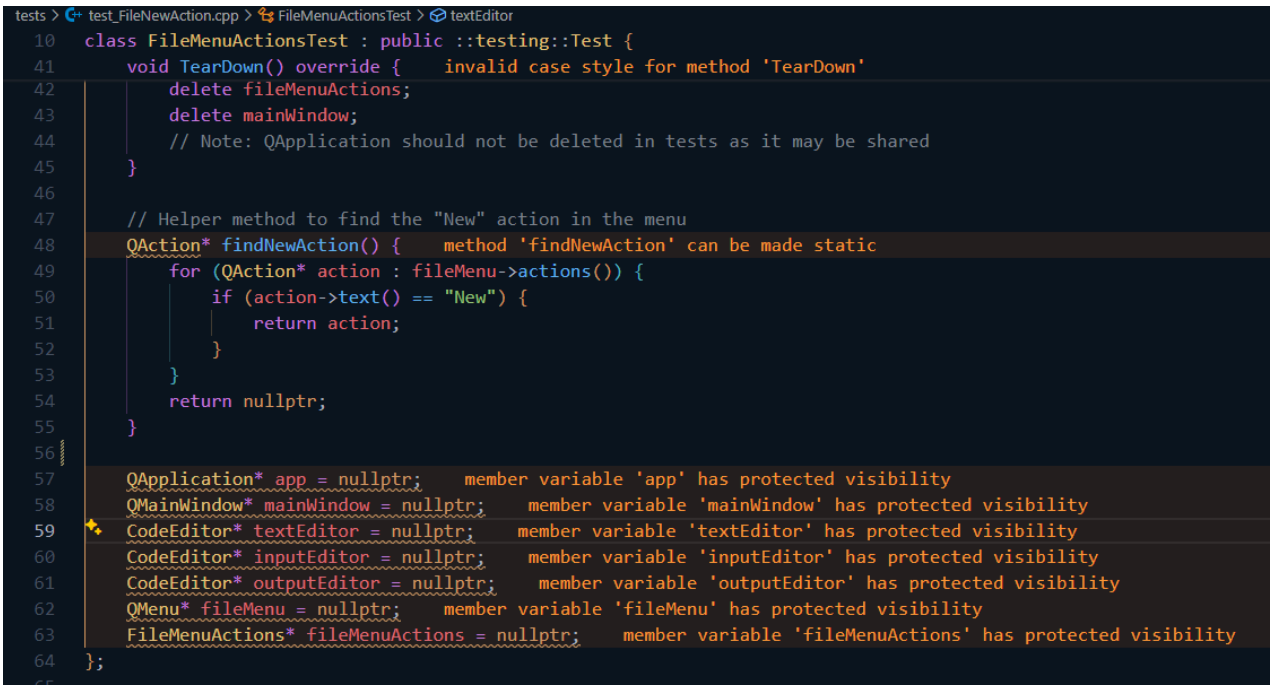
Before:

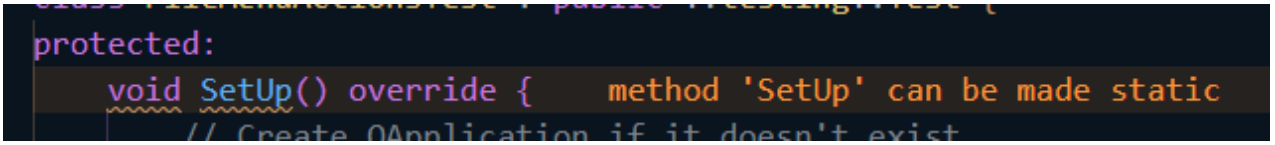
```
89
90    for (int i = 0; i < 10; ++i)    10 is a magic number; consider replacing it with a named constant
91    {
92
93    }
94
```

Alt + Shift + F After:

```
for (int i = 0; i < 10; ++i) {    10 is a magic number; consider replacing it with a named constant
}
```

clang-tidy:


tests > test_FileNewAction.cpp > FileMenuActionsTest > textEditor
class FileMenuActionsTest : public ::testing::Test {
void TearDown() override { invalid case style for method 'TearDown'
delete fileMenuActions;
delete mainWindow;
// Note: QApplication should not be deleted in tests as it may be shared
}
// Helper method to find the "New" action in the menu
QAction* findNewAction() { method 'findNewAction' can be made static
for (QAction* action : fileMenu->actions()) {
if (action->text() == "New") {
return action;
}
}
return nullptr;
}
QApplication* app = nullptr; member variable 'app' has protected visibility
QMainWindow* mainWindow = nullptr; member variable 'mainWindow' has protected visibility
CodeEditor* textEditor = nullptr; member variable 'textEditor' has protected visibility
CodeEditor* inputEditor = nullptr; member variable 'inputEditor' has protected visibility
CodeEditor* outputEditor = nullptr; member variable 'outputEditor' has protected visibility
QMenu* fileMenu = nullptr; member variable 'fileMenu' has protected visibility
FileMenuActions* fileMenuActions = nullptr; member variable 'fileMenuActions' has protected visibility
};


protected:
void SetUp() override { method 'SetUp' can be made static